Authored by: Arpit Raj, LNMIIT Jaipur
Date: Sunday, October 25, 2026
© 2026 Arpit Raj. All rights reserved.

C++ Runtime Polymorphism: Virtual Functions & VTables

Function Overriding Cases

-> overriding occurs when a derived class provides a matching function for a VIRTUAL FUNCTION in base class.

```
class A {
public:
    virtual void show() {
        cout << "A";
    }
};

class B : public A {
public:
    void show() override {
        cout << "B";
    }
};
```

now if we do `B obj` and `obj.show()` -> O/p = B

Case 1: If not virtual function and no override

```
B obj;
A* ptr = &obj;
ptr->show();
```

since `show` not virtual, `ptr` is a pointer of type `A*`, so function is selected using pointers type, not actual objects type (type B) -> O/p = A

Case 2: if virtual function used

then selection is based on actual objects type and not on pointer type. so now same code outputs B.

Basically: if base class and derived class have function of same name and it is not a virtual function then

Without virtual:

Base pointer -> Base function

even if the actual object is derived, the non virtual call is selected using pointer's
static type

.

With virtual:

Base pointer -> Derived override

actual object's type determines
final overrider

at runtime.

Hence

```
class A {
public: virtual void show() { }
};

class B {
public: void show() override { }
};
```

```
B obj;
A* ptr = &obj;
ptr->show();
```

will now show B.

What is Polymorphism?

-> polymorphism means many forms, here the same function can produce different behaviours.

- Base class gives generalized behaviour
- derived class gives specialized behaviour

Eg let there be a `sound()` function: general base class -> `Animal`.
then same sound function can do:

> - Animal sound -> base class
> - Cat meows -> derived class
> - dog barks -> derived class

Rules

- virtual is declared in Base class
- Matching derived class function overrides virtual function
- use `override` in derived class to verify override
- runtime polymorphism works through base class pointer
- non virtual function -> call resolved -> using expression **static type**
- virtual function -> call resolved -> using objects **final overrider**

Internal Working: VTable & VPtr

- the keyword `virtual` tells the compiler that calls to a function should support dynamic dispatch
- to implement this, compilers use 2 mechanism

> **MECHANISMS OF DYNAMIC DISPATCH**
>
> - **Vtable:** table of function address associated with class's virtual function
> - **Vptr:** a hidden pointer in polymorphic object pointing to Vtable

- A's virtual table: `show -> A::show();`
- B's virtual table: `show -> B::show();` (because `B::show()` overrides `A::show()`, function address points to `B::show`)

Step-by-Step: What Happens Under the Hood

WHAT happens

S1) `B obj` -> It inherits from A. (Diagram: [A part | B part])

S2) `A* ptr = &obj` -> creates a pointer of type `A*`. `&obj` -> gets the address of A part within object B (here it is simply the address of B object).

S3) `ptr->show()` -> since show is declared virtual in A, we respect runtime polymorphism. actual object is B. B overrides `show()` in Vtable. -> O/p = B.

© Arpit Raj, LNMIIT Jaipur

Name Hiding vs Overriding

```
class A {
public: virtual void show() { }
};

class B {
public: void show(int x) { }
};
```

now if we do:

```
B obj;
obj.show();
```

this wont compile because `B::show(int x)` hides `A::show()` from scope class of B.

How ?

S1) compiler find `void show(int x)` in B

S2) once compiler finds a matching name in current scope, it stops searching up the hierarchy

S3) now the compiler matches argument, expectations: `int`, reality: none. hence compile error

However

```
B obj;
A* ptr = &obj;
ptr->show();
```

will work and print A.

HOW

S1) look for `show()` inside class A

S2) finds `virtual void show()` with no parameters

S3) since marked virtual, program checks dynamic type. but since `B::show(int x)` have a different signature it does not override `A::show`, so A is printed.